Лабораторна робота 5

АДАПТИВНІСТЬ, КРОСБРАУЗЕРНІСТЬ ТА МЕТРИКИ ЕФЕКТИВНОСТІ ФРОНТЕНДУ

Мета – набути практичних навичок у реалізації адаптивності фронтенду, методів та підходів, які забезпечують його кросбраузерність, а також в оцінці ефективності розробленого продукту.

Кількість годин – 2

Теоретичні відомості

Розробка фротенду програмних систем тісно пов'язана з його тестуванням. Саме цей етап життєвого циклу продукту забезпечує його адаптивність та кросбраузерність, а також передбачає його рефакторинг.

1. Адаптивність фронтенду:

Адаптивність фронтенду – це його здатність автоматично підлаштовуватись під різні розміри екранів та пристроїв, на яких він переглядається. Відповідність розробленого продукту вимогам адаптивності включає оптимізацію дизайну, швидкості завантаження, розміщення елементів і загального користувацького досвіду.

Існує два основні підходи до реалізації адаптивності:

- responsive web design (RWD) або чутливий дизайн;
- adaptive web design (AWD) або адаптивний дизайн.

Основна відмінність чутливого й адаптивного дизайну — у реакції на зміну розміру екрана. Чутливий дизайн допомагає динамічно підлаштовувати контент під розміри та роздільну здатність екрана для одного й того ж макета, адаптивний — підбирати найбільш відповідний із декількох макетів і відображати його користувачеві.

На сьогодні, частіше використовується чутливий дизайн, оскільки створення та підтримання єдиного універсального макета займає менше часу й коштує дешевше.

Адаптивний дизайн виправданий у складних проєктах, заточених під певні пристрої: наприклад, під час розроблення сайту онлайн-банкінгу, більшість користувачів якого відвідують сайт з останніх моделей iPhone.

1.1. Responsive web design (RWD) або чутливий дизайн:

Чутливий вебдизайн базується на трьох принципах:

• гнучкий макет на основі сітки (flexible, grid-based layout);

• гнучкі зображення (flexible images);

• медіазапит (CSS3 media queries).

Гнучкість макета базується на використанні відносних одиниць вимірювання замість фіксованих піксельних значень. Проблема гнучких зображень вирішується за допомогою правила img (width: 100%; max-width: 100%;) для всіх картинок на сайті. Це правило гарантує, що зображення ніколи не будуть ширше, ніж їхні контейнери і ніколи не перевищать своїх справжніх розмірів на великих екранах. У 2022 році в CSS з'являються наступні одиниці:

• lvh (Large Viewport Height) — ця одиниця відображає повну висоту екрана, коли інтерфейс браузера не впливає на доступну висоту;

• svh (Small Viewport Height) — на відміну від lvh, ця одиниця вимірює мінімальну можливу висоту екрану, коли інтерфейс браузера займає більше простору. Наприклад, при першому завантаженні сторінки на мобільному телефоні, коли видимий адресний рядок ще не прихований. Вона корисна для тих випадків, коли необхідно адаптувати дизайн до ситуації, де доступної висоти менше.

• dvh (Dynamic Viewport Height) — це динамічна одиниця, яка враховує зміни в інтерфейсі браузера в режимі реального часу. Вона свого роду динамічне поєднання двох попередніх. Наприклад, коли користувач взаємодіє з екраном, і адресний рядок зникає або з'являється, dvh автоматично підлаштовує висоту елементів під ці зміни.

Гнучкі зображення необхідні, щоб підібрати оптимальний варіант для конкретного пристрою залежно від розміру, орієнтації та типу дисплея.

Для цього використовуються атрибути srcset і sizes: перший дає можливість вказати шляхи до всіх варіацій зображення та розміри в пікселях, другий — задавати критерії вибору того чи іншого зображення через медіазапит.

За допомогою атрибутів можна підказати браузеру, яку версію картинки завантажувати: для вузького екрана — полегшену версію для смартфонів, для широкого телевізора — картинку в потрійному розмірі.

Медіазапити змінюють стилі на підставі характеристик пристрою, пов'язаних з відображенням контенту, включаючи тип, ширину, висоту, орієнтацію й характеристики екрану. Це правила CSS, які дозволяють керувати стилями елементів залежно від значень технічних параметрів пристроїв. Браузер застосовує таблицю стилів тільки в разі, якщо активувався саме цей тип пристрою.

Загалом медіа-запит складається з ключового слова, що описує тип пристрою (необов'язковий параметр) і вираз, котрий перевіряє характеристики цього пристрою. З усіх характеристик найчастіше перевіряється ширина пристрою width. Медіа-запит є логічним виразом, який повертає істину або хибність.

Запит записують у кінці таблиці стилів (у стилів, які записані пізніше, вища пріоритетність).

Основні типи носіїв у медіа-запитах: all – використовується за замовчуванням; handheld – смартфони та аналогічні пристрої; print – принтери та інші друкарські пристрої; projection – проектор; screen – екран монітора; speech – мовні синтезатори; tv – телевізори.

Список популярних медіа-запитів для стандартного набору розширення екранів:

```
@media only screen and (max-width : 1920px) {/* CSS правила */ }
@media only screen and (max-width : 1680px) {}
@media only screen and (max-width : 1366px) {}
@media only screen and (max-width : 1280px) {}
@media only screen and (max-width : 1024px) {}
```

```
@media only screen and (max-width : 800px) {}
@media only screen and (max-width : 768px) {}
@media only screen and (max-width : 600px) {}
@media only screen and (max-width : 533px) {}
@media only screen and (max-width : 360px) {}
@media only screen and (max-width : 320px) {}
@media only screen and (max-width : 240px) {}
@media only screen and (max-width : 176px) {}
```

Наприклад, якщо ви хочете створити макет із трьома колонками для великих екранів та макет з однією колонкою для малих екранів (наприклад, телефонів), ви можете змінити напрямок flex-напрямку з рядка на колонку в певній точці зупинки (600 пікселів у прикладі нижче):

```
.flex-container {
 display: flex;
 flex-direction: row;
}

.flex-item {
 padding: 10px;
 font-size: 30px;
 text-align: center;
 width: 100%;
}

/* Make a one column-layout instead of three-column layout */
@media (max-width: 600px) {
 .flex-container {
 flex-direction: column;
 }
}
```

1.2. Adaptive web design (AWD) або адаптивний дизайн:

Його основою є декілька фіксованих макетів найпопулярніших розмірів екрана. Усі варіанти фронтенду розгортаються за одним і тим самим URL.

Зазвичай розробляють шість макетів відповідно до найпопулярнішої ширини екрана: 320 px, 480 px, 760 px, 960 px, 1200 px, 1600 px.

Головне в адаптивній верстці — це прив’язка до конкретних пристроїв з певною роздільністю екрану.

Стилі перемикаються від одного брейкпоінта (контрольні точки, сталий термін у програмуванні, у випадку верстки означає перемикання з одних умов на інші) до іншого. Тобто насправді існують фіксовані макети для кожного девайсу.

Деякі інструменти для адаптивної верстки:

• "Media Query Inspector" дає змогу розробникам перевіряти та регулювати медіа-запити в реальному часі, забезпечує точне налаштування відображення контенту на пристроях будь-якого типу;

"Responsive Design Mode" в інструментах розробника браузера також стає незамінним. Він надає прев'ю для різних розмірів екранів, що дає змогу миттєво оцінювати, який вигляд сайт має на мобільних ґаджетах і планшетах.

Метрики ефективності фронтенду охоплюють багато аспектів розробки. Зокрема, продуктивність (швидкість завантаження та взаємодії), якість коду (читабельність, підтримка, баги) та UX/UI (зручність користувача, конверсії), включаючи швидкість (Time to Interactive), використання ресурсів (пам'ять, CPU), кількість помилок у консолі та показники Google Core Web Vitals, що разом допомагають покращити досвід користувача та роботу програмної системи.

2. Кросбраузерність:

Кросбраузерність — це правильна верстка фронтенду, за допомогою якої він однаково відображаються в різних браузерах та їх версіях. Це пов’язано зокрема з відсутністю єдиного стандарту при розробці браузерів. Тому, кожний переглядач має свої відмінності в інтерпретації гіпертексту, що пов'язано з різною реалізацією рушіїв зображення та створенням пропрієтарних тегів для розширення можливостей стандартного HTML. Так, один текст може бути відтворено по-різному в Internet Explorer та Safari.

Забезпечення кросбраузерності відбувається за допомогою HTML і CSS, а також за допомогою різноманітних хаків, а в окремих випадках — JavaScript. Вона досягається декількома способами:

- HTML використовує лише ті елементи, які відображаються у всіх браузерах однаково. Потрібно віддавати перевагу універсальним елементам — тим, які однаково відображаються у більшості браузерів;
- хаками – набором спеціальних правил та селекторів, який розуміє певний браузер;
- реалізацією на основі готових фреймворків та бібліотек.

У кожного браузера є свої вбудовані, експериментальні, або нестандартні властивості, для коректної роботи яких, використовують вендорні префікси. Такі властивості змінюють поведінку елемента тільки в певному браузері та ігноруються іншими платформами.

Перелік вендорних префіксів для різних браузерів:

- -webkit- для Chrome, Safari, Opera, Microsoft Edge;
- -moz- для Firefox;
- -o- для ранніх версій Opera;
- -ms- для ранніх версій Microsoft Edge (до 18 версії) та Internet Explorer.

Наприклад, елемент border-radius у MozillaFirefox представлений властивістю -moz-border-radius, а в Chrome і Safari— -webkit-border-radius. Наведемо приклад уніфікації властивості елемента мати тінь від рамки:

```
.element_with_border_shadow {
    -webkit-box-shadow: 0 0 4px rgba (0, 0, 0, 0.5);
    -moz-box-shadow: 0 0 4px rgba (0, 0, 0, 0.5);
    -ms-box-shadow: 0 0 4px rgba (0, 0, 0, 0.5);
    box-shadow: 0 0 4px rgba (0, 0, 0, 0.5);
```

}

Як відомо, кожен браузер за замовчуванням має певний набір базових стилів, які він застосовує до сторінки. У різних браузерах ці правила трохи відрізняються. Щоб їх усунути та зробити за замовчуванням відображення сторінки у всіх браузерах однаковим, використовують спеціальні CSS-файли: reset.css або normalize.css. Файл reset.css містить перелік усіх можливих HTML-тегів і скидає їх значення в нуль, тобто прибирає всі можливі відступи, робить шрифт однаковим у всіх тегах. Таким чином, всі заголовки й абзаци відображаються простим текстом, одним розміром і без відступів. Як наслідок відбувається скидання стилів за замовчуванням у всіх браузерах. Спочатку на сторінці користувачу слід підключити файл reset.css, а потім власний файл зі стилями style.css. У будь-якому браузері вся розмітка ґрунтуватиметься на тих стилях, які потрібно поставити в style.css.

Normalize.css — продукт глибокого дослідження відмінностей між початковими стилями браузера. Дослідження провів Ніколас Галахер, взявши за мету: зберігати корисні налаштування; нормалізувати стилі для більшості HTML-елементів; коригувати помилки й основні невідповідності браузера; удосконалювати юзабіліті непомітними поліпшеннями; пояснювати код, використовуючи коментарі та детальну документацію.

Наразі normalize.css використовується в Twitter Bootstrap, HTML5 Boilerplate, GOV.UK, Rdio, CSS Tricks і в багатьох інших фреймворках, інструментах, тощо.

3. Метрики продуктивності (Performance Metrics):

LCP (Largest Contentful Paint) — це показник швидкості завантаження, що вимірює час від початку завантаження сторінки до моменту, коли браузер повністю відобразить найбільший видимий елемент контенту (наприклад, велике зображення, відео чи текстовий блок), а ідеально – цей час має бути меншим за 2.5 секунди, щоб користувач не сприймав сторінку як повільну, що покращує досвід та позиції в пошуку Google.

FID (First Input Delay) / INP (Interaction to Next Paint): швидкість реакції на першу взаємодію користувача (клік, тап). є показником, який оцінює загальну інтерактивність сторінок сайтів на основі часу реакції на взаємодію користувача. INP є результатом спостереження за всіма кваліфікованими взаємодіями, що відбуваються протягом візиту користувача на сторінку.

Метрика Interaction to Next Paint призначена для оцінки швидкості відгуку на дії користувача. Тож вона передбачає вимірювання затримки введення: від моменту взаємодії користувача зі сторінкою до появи візуальної відповіді. INP реєструє затримку під час взаємодії з вебсторінкою, навіть коли відбувається її перезавантаження. На думку розробників Google, швидкість відгуку має вирішальне значення, адже людина має бачити, що сайт реагує на взаємодію з нею.

CLS (Cumulative Layout Shift): стабільність верстки (відсутність "стрибків" елементів). Вона вимірює неочікувані проблеми візуальної стабільності на веб-

сторінках, де контент перескакує під час завантаження сторінки, дратуючи користувачів і потенційно спричиняючи випадкові кліки.

Час до інтерактивності (Time to Interactive, TTI) — це метрика веб-продуктивності, що визначає момент, коли веб-сторінка стає повністю функціональною та здатною реагувати на дії користувача (кліки, введення тексту). Це відбувається після відображення корисного контенту (FCP) та реєстрації обробників подій для видимих елементів, при цьому браузер не виконує довгих завдань і може миттєво відповідати на взаємодії, що покращує досвід користувача та SEO.

Час завантаження JavaScript: як швидко виконується JS. Ідеально, коли він не перевищує 1-2 секунди для швидкого досвіду користувача. Для вимірювання використовують інструменти розробника в браузері, API Performance, а оптимізація включає відкладене завантаження, мініфікацію та асинхронні запити.

Розмір бандла (Bundle Size): загальний розмір усіх файлів (HTML, CSS, JavaScript, зображень), зібраних разом для веб-сторінки чи застосунку, і його зменшення критично важливе для швидкості завантаження, продуктивності та позитивного користувацького досвіду, оскільки більші бандли сповільнюють сайт, вимагають більше трафіку та ресурсів. Цей розмір включає код компонентів, стилі та залежності, і оптимізується за допомогою технік трейшікінгу (treeshaking), лінивої загрузки (lazy loading), оптимізації імпорту та видалення невикористовуваного коду.

Використання пам'яті та CPU: навантаження на пристрій користувача.

3.1. Метрики якості коду та розробки:

Щільність дефектів (Defect Density): ключова метрика якості ПЗ, що показує кількість знайдених багів (дефектів) на певну одиницю продукту, найчастіше – на 1000 рядків коду (KLOC) або на функцію/модуль, яка допомагає оцінити стабільність коду та ефективність тестування, вказуючи на проблеми в певних частинах системи чи загальну якість розробки.

Цикломатична складність (Cyclomatic Complexity): вимірює складність логіки функцій чи методів, показуючи кількість незалежних шляхів виконання.

Час виправлення багів (Bug Fix Time): скільки часу займає усунення дефекту.

Розмір коду (LOC, Lines of Code): відноситься до загальної метрики, що використовується для вимірювання розміру класу, функції або загальної кількості рядків усієї реалізації. Вона кількісно визначає кількість рядків коду в програмі або програмному забезпеченні.

Метрики зв'язності (Coupling) та згуртованості (Cohesion): вимірюють, наскільки залежні класи між собою та наскільки функціонально об'єднані елементи класу.

3.2. Метрики UX та бізнесу:

Коефіцієнт конверсії (Conversion Rate): показник ефективності в маркетингу, що показує відсоток користувачів, які виконали цільову дію (купівлю, реєстрацію, підписку, заповнення форми) з-поміж загальної кількості

відвідувачів або взаємодій. Розраховується за формулою: (Кількість цільових дій / Загальна кількість відвідувачів) * 100%. Цей метрика допомагає оцінити ефективність сайту та рекламних кампаній, дозволяючи збільшувати прибуток без збільшення витрат на трафік.

Час на сайті / глибина перегляду: як довго користувач залишається на сайті та скільки сторінок переглядає.

Показник відмов (Bounce Rate): відсоткове співвідношення кількості відвідувачів, що покинули сайт прямо зі сторінки входу або переглянули не більше однієї сторінки сайту.

Задоволеність користувачів (NPS, CSAT): опитування для оцінки загального досвіду. NPS (Net Promoter Score) оцінює лояльність та готовність рекомендувати бренд (від -100 до +100), та CSAT (Customer Satisfaction Score), яка вимірює задоволеність конкретною взаємодією чи продуктом (зазвичай за шкалою 1-5 або 1-10, виражається у відсотках). CSAT фокусується на поточних транзакціях, даючи швидкий зворотний зв'язок, тоді як NPS вимірює довгострокову відданість і є прогностичною метрикою лояльності.

Завдання

1. Вивчити основні прийоми реалізації гнучкого та адаптивного дизайну фронтенду програмних систем, способи досягнення кросбраузерності фронтенду та ознайомитись з функціоналом інструментів для вимірювання ефективності фронтенду: Google PageSpeed Insights, Lighthouse, Google PageSpeed Insights, Lighthouse, Chrome DevTools, WebPageTest, Sentry, New Relic, Datadog;

2. Провести перевірку адаптивності реалізованого у лабораторних роботах 1-4 фронтенду для стандартного набору розширень екранів пристроїв, забезпечивши зручний користувацький досвід у навігації, перегляді меню та каталогів, а також заповненні форми зворотнього зв’язку, за допомогою інструментів розробника. Наприклад, можна скористатись F12, що у більшості браузерів відкриває front-end development/debugging. За необхідності внести зміни у код: замінити абсолютні одиниці виміру на відносні (див. Лабораторна робота 2); всі зображення зробити гнучкими; забезпечити наявність медіа-запитів для різних типів розширень екранів; реалізувати брейкпоінт-макети. Сформувати таблицю, що відображає отримані реазультати;

3. Провести перевірку кросбраузерності реалізованого у лабораторних роботах 1-4 фронтенду, сформуваши таблицю відповідності між використаними тегами та їх підтримкою браузерами Chrome, Edge/Internet Explorer, Firefox, Safari та Opera, використовуючи довідник з підтримки тегів різними браузерами. За необхідності внести зміни у код за допомогою вендорних префіксів у реалізації стилістики тих елементів, властивості яких відрізняються у Chrome, Edge/Internet Explorer, Firefox, Safari та Opera [5];

4. Провести заміри основних метрик ефективності програмного продукту розробленого у роботах 1-4 та оформити відповідно до них таблиці, які відображають отримані результати. Провести аналіз причин наявності низьких показників ефективності.

Контрольні запитання

1. Чи є надмірне використання стилів кроком на зустріч адаптивності?
2. Чи попередній перегляд вигляду фронтенду на емуляторах повністю забезпечує адаптивність?
3. Чим відрізняється адаптивність від доступності?
4. Перелічіть проблеми, які виникають через відмінності у браузерах?
5. Яка основна причина наявності відмінностей між різними браузерами?
6. Яка роль HTML5 у забезпеченні кросбраузерності фронтенду?
7. Як досягнути кросбраузерності фронтенду, використовуючи правила CSS?
8. Які фреймворки спрощують розробку кросбраузерного фронтенду програмної системи?
9. Проведіть огляд основних класів метрик ефективності фронтенду.
10. Які прийоми запобіганню поганих метричних показників?
11. Які інструменти для вимірювання метрик ефективності фронтенду?
12. Яка роль фремворків та бібліотек для розробки фронтенду у покращені метрик його ефективності?

ЛІТЕРАТУРА: **[7-10, 19, 20].**

СПИСОК ЛІТЕРАТУРИ

1. Мельник Р. Програмування веб-застосувань (фронт-енд та бек-енд). — Львів: Львівська політехніка, 2018. — 250 с.
1. Двірничук К.В., Вацек Д.О. Веб-програмування та веб-дизайн: навч. посіб. — Чернівці : Чернівец. нац. ун-т ім. Ю. Федьковича. — 2022. — 472 с.
2. Duckett J. HTML & CSS: Design and Build Web Sites. — O'Reilly, 2011. — 512 pp.
3. Robson E., Freeman E. Head First HTML and CSS: A Learner's Guide to Creating Standards-Based Web Pages. — O'Reilly, 2012. – 762 pp.
4. HTML Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/html/
5. CSS Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/css/
6. Responsive web design basics [Електронний ресурс]. Режим доступу: https://web.dev/articles/responsive-web-design-basics
7. Top 10 Front-End Frameworks for Responsive Design 2024 [Електронний ресурс]. Режим доступу: https://daily.dev/blog/top-10-front-end-frameworks-for-responsive-design-2024
8. Robson E., Freeman E. Head First HTML and CSS: A Learner's Guide to Creating Standards-Based Web Pages. — O'Reilly, 2012 – 762 pp.
9. Довідник з підтримки тегів різними браузерами [Електронний ресурс]. Режим доступу: https://www.w3schools.com/tags/ref_html_browsersupport.asp
10. Flanagan D. JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language. — O'Reilly, 2020. — 704 pp.
11. McMillan M. Data Structures and Algorithms with JavaScript: Bringing classic computing approaches to the Web. — O'Reilly, 2014. — 370 pp.
12. JavaScript Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/js/
13. Resources for developers, by developers. MDN Web Docs [Електронний ресурс] – Режим доступу: https://developer.mozilla.org/en-US/docs/Web
14. React Tutorial [Електронний ресурс]. Режим доступу: https://www.w3schools.com/js/
15. Vue Tutorial Електронний ресурс]. Режим доступу: https://vuejs.org/guide/quick-start.html
16. Next Tutorial [Електронний ресурс]. Режим доступу: https://nextjs.org/
17. Angular Tutorial [Електронний ресурс]. Режим доступу: https://angular.dev/
18. Aluri Y. S., Frontend Performance Optimization of Large-Scale E-commerce Landing Pages: A Comprehensive Analysis // International Journal of Computational and Experimental Science and ENgineering (IJCESEN). — Vol. 11 (4). — 2025. — pp. 9123-9134.
19. Fauzan P., Hasbullah, Farhan M., Reni P. Technical Performance Comparison of Modern Frontend frameworks Study on Svelte, React, and Vue // Brilliance: Research of Artificial Intelligence — Vol. 5 (1). — May 2025.